Web Application Security Assessment Report

Candidate Information

Name : G.M. Kavishka Subhashi Rathnasena

Email : kaviisubhashi@gmail.com

Contact Number : 071 36 66086

Assessment Information

Position Applied: Intern – Cyber Security

Date : 24/3/2026

Target and Environment

Target Application : DVWA

Testing Environment : Localhost (http://localhost:8080)

Operating System : Windows

Testing Tool : Burp Suite Community Edition

1.Introduction

This assessment focuses on performing a security analysis of a deliberately vulnerable web application, **Damn Vulnerable Web Application (DVWA)**. The objective is to identify, analyze, and demonstrate common web application vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), authentication issues, Insecure Direct Object References (IDOR), and security misconfigurations.

By using industry-standard tools and techniques, this assessment aims to develop practical skills in penetration testing, understand real-world attack scenarios, and propose effective mitigation strategies to enhance application security. All testing is conducted in a controlled and ethical environment for educational purposes.

2. Scope

The scope of this assessment was limited to locally deployed instance of the DVWA. Testing was conducted in a controlled environment to ensure no impact on external systems.

Application : DVWA

Environment : Localhost (http://localhost:8080)

The assessment focused on identifying vulnerabilities within the application's web interface.

3. Methodology

The assessment follows a structured penetration testing methodology to systematically identify and analyze vulnerabilities in DVWA.

3.1. Environment Setup

- Install and configure DVWA using Docker, XAMPP, or a local server
- Set security level (Low/Medium) for testing purposes
- Configure database and ensure the application is running properly

3.2. Reconnaissance and Application Understanding

- Explore DVWA modules and functionalities
- Identify input fields, parameters, and user interaction points
- Understand authentication and session behavior

3.3. Vulnerability Identification

- Test different modules to identify common vulnerabilities:
  - SQL Injection in input fields
  - XSS through user inputs
  - Weak authentication mechanisms
  - IDOR by manipulating parameters
  - Misconfigurations in application settings

3.4. Security Testing Using Tools

- Use Burp Suite to:
  - ✓ Intercept HTTP requests and responses
  - ✓ Modify parameters and headers
  - ✓ Replay and analyze requests
- Observe how the application responds to manipulated inputs.

3.5. Exploitation

- Demonstrate each identified vulnerability
- Capture screenshots and proof of exploitation
- Show how attackers can misuse vulnerabilities

3.6. Risk Analysis

- Assign severity levels (Low, Medium, High, Critical)
- Analyze the potential impact of each vulnerability

- Describe possible real-world attack scenarios

3.7. Mitigation and Recommendations

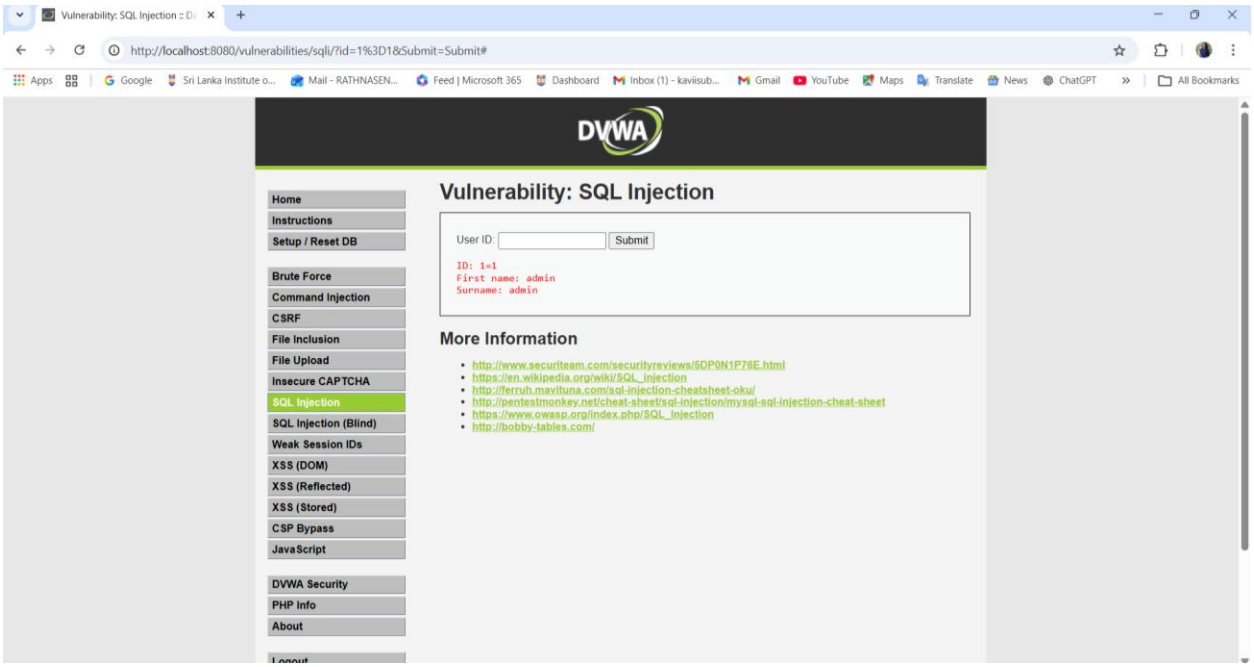
- Suggest appropriate fixes for each vulnerability
- Provide secure coding practices and configuration improvements
- Recommend best practices such as input validation, proper authentication, and secure configurations

4. Findings

During the assessment of the DVWA application, multiple vulnerabilities were identified. These vulnerabilities allow attackers to manipulate input, gain unauthorized access, or retrieve sensitive information.

4.1 SQL Injection

- The application fails to properly validate user input in form fields, allowing malicious SQL queries to be executed directly on the database.



In this specific view, the "User ID" input field has been exploited to bypass standard database queries. Here is a breakdown of what is happening:

- **The Payload:** The URL shows id=1%3D1. In plain text, this is 1=1.
- **The Logic:** By adding OR 1=1 (a statement that is always true), the attacker tricks the database into ignoring the specific ID requested and instead returning records from the table.
- **The Result:** You can see that the application has returned data for the "admin" user. In many cases, this technique can be used to dump an entire database of usernames, passwords, or personal info.

Impact

- Unauthorized access to user accounts
- Exposure of sensitive data
- Potential full system compromise

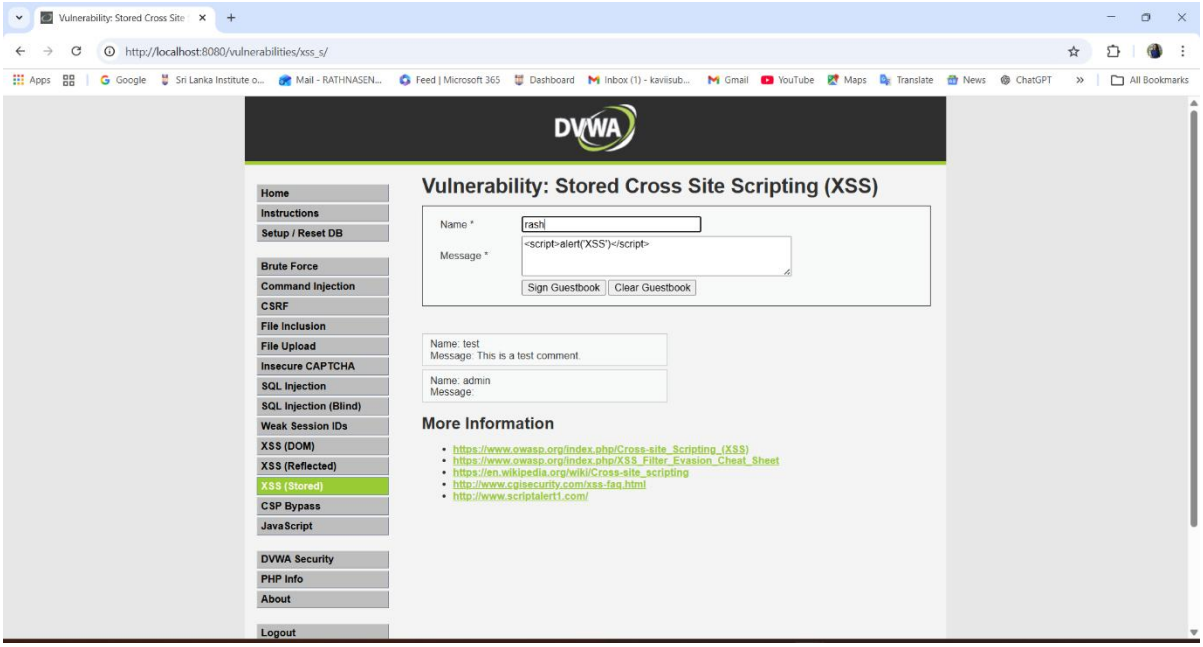
Severity

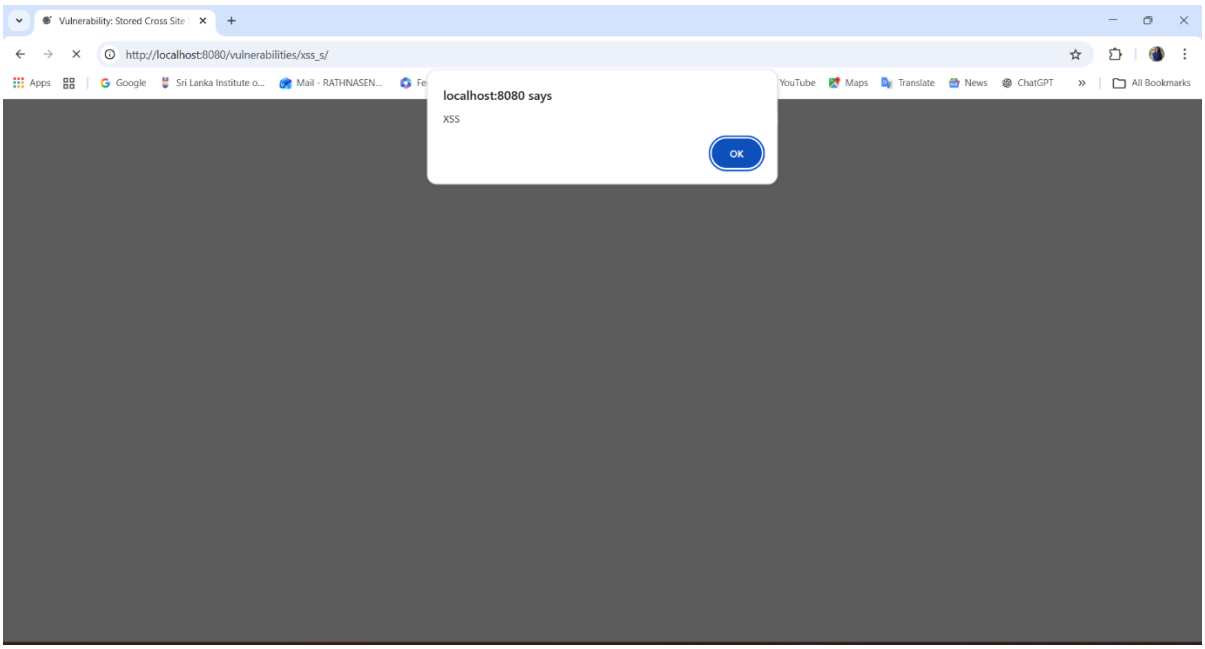
Critical

Mitigation

- Avoid dynamic SQL queries.
- Use parameterized queries or prepared statements.
- Validate and sanitize all user inputs.

4.2 Stored Cross Site Scripting





The screenshots demonstrate a **Stored Cross-Site Scripting (XSS)** vulnerability within the DVWA guestbook, where a malicious JavaScript payload, <script>alert('XSS')</script>, is submitted and permanently saved into the application's database. Unlike other forms of XSS, this "stored" version is particularly dangerous because the script executes automatically in the browser of any user who views the guestbook page, as seen by the triggered alert box

Impact

- Theft of sensitive user information (session cookies, credentials).
- Account takeover of affected users.
- Defacement of web content.
- Spread of malware to site visitors.

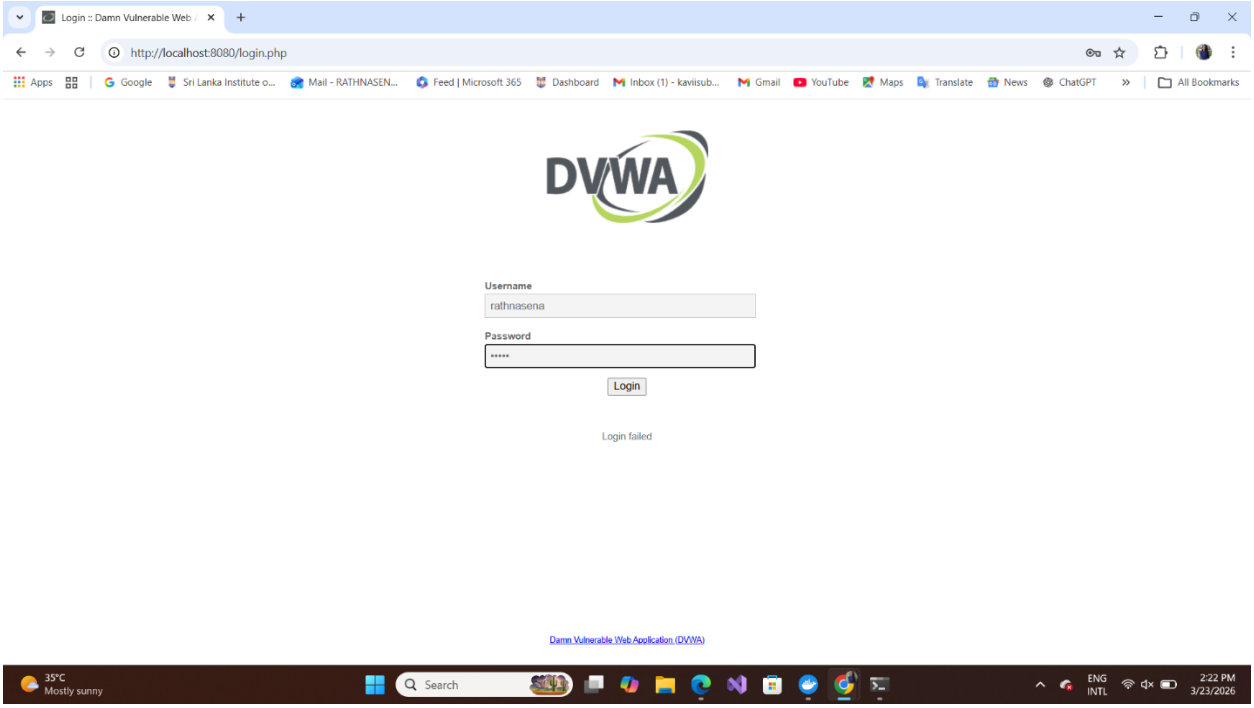
Severity

High

Mitigation

- Properly encode output before rendering user input in web pages.
- Sanitize all input using a whitelist approach.
- Implement input validation both on the client and server sides.
- Avoid directly storing raw HTML or script content submitted by users.

4.3 Auth Issue (Brute Force)



The application is vulnerable to brute-force attacks due to the absence of proper authentication protection mechanisms. During testing, multiple login attempts were performed using different user name and password combinations without any restrictions. Tried 10 attempts still no restrictions.The system did not enforce account lockout policies, introduce delays between failed attempts, or implement CAPTCHA verification.

As a result, an attacker can repeatedly attempt different password combinations for a valid username until the correct credentials are discovered. This significantly increases the risk of unauthorized access, especially if weak or commonly used passwords are in use.

Impact

If authentication is "broken" or lacks rate-limiting, an attacker can gain unauthorized access to accounts, leading to data breaches or administrative takeover.

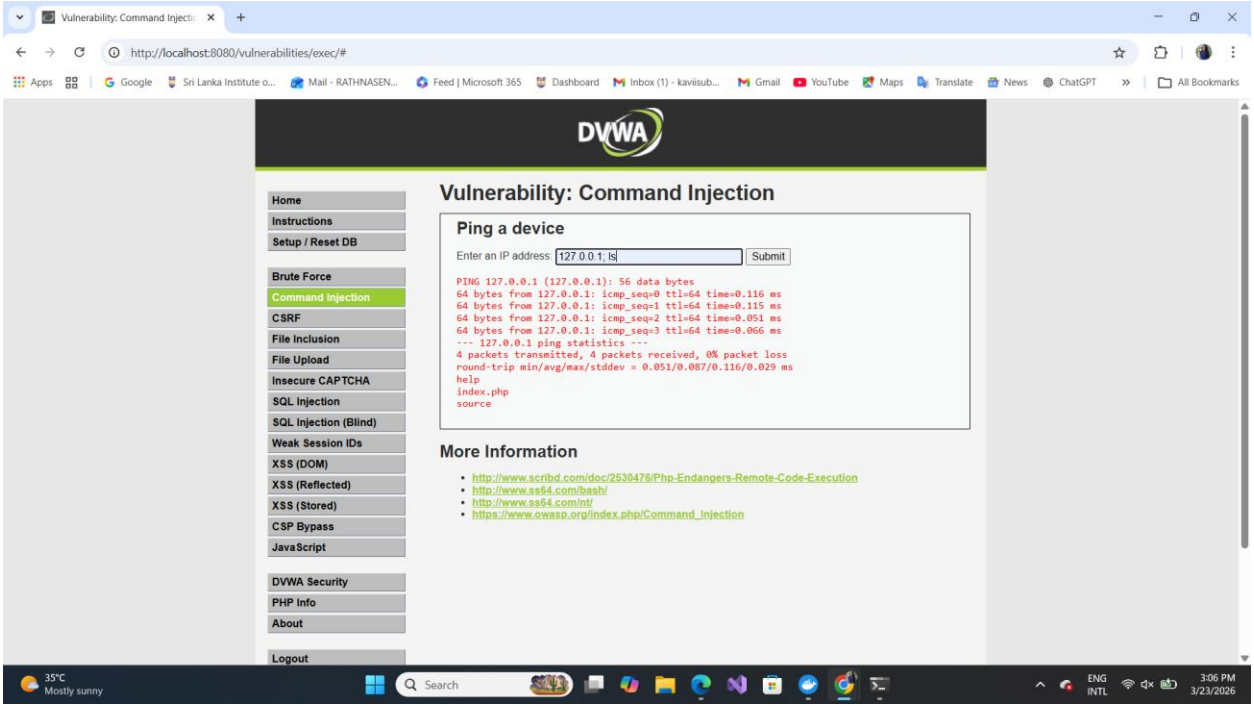
Severity

High

Mitigations

- **Account Lockout Policy:** Temporarily lock an account after a set number of failed attempts (e.g., 5 attempts).
- **Rate Limiting:** Implement delays between login requests to slow down automated brute-force tools.
- **Generic Error Messages:** Use non-specific messages like "Invalid login credentials" to prevent username enumeration.
- **Multi-Factor Authentication (MFA):** Add a second layer of security so that a password alone is not enough to gain access.
- **CAPTCHA:** Use visual challenges to ensure the login attempt is being made by a human rather than a bot.

4.4 Command Injection



In the "Ping a device" tool, the intended use is to enter an IP address to run a standard ping command. However, the user entered 127.0.0.1; ls

- **The Exploit:** The semicolon (;) acts as a command separator in Linux/Unix.
- **The Result:** The server first executes the ping command and then immediately executes the ls command. You can see the file list (help, index.php, source) printed at the bottom of the output box. This proves the attacker can run arbitrary commands on the host operating system.

Impact

An attacker can gain full control over the server. They can read sensitive files (like /etc/passwd), delete data, install backdoors, or use the server to launch attacks on other systems in the internal network.

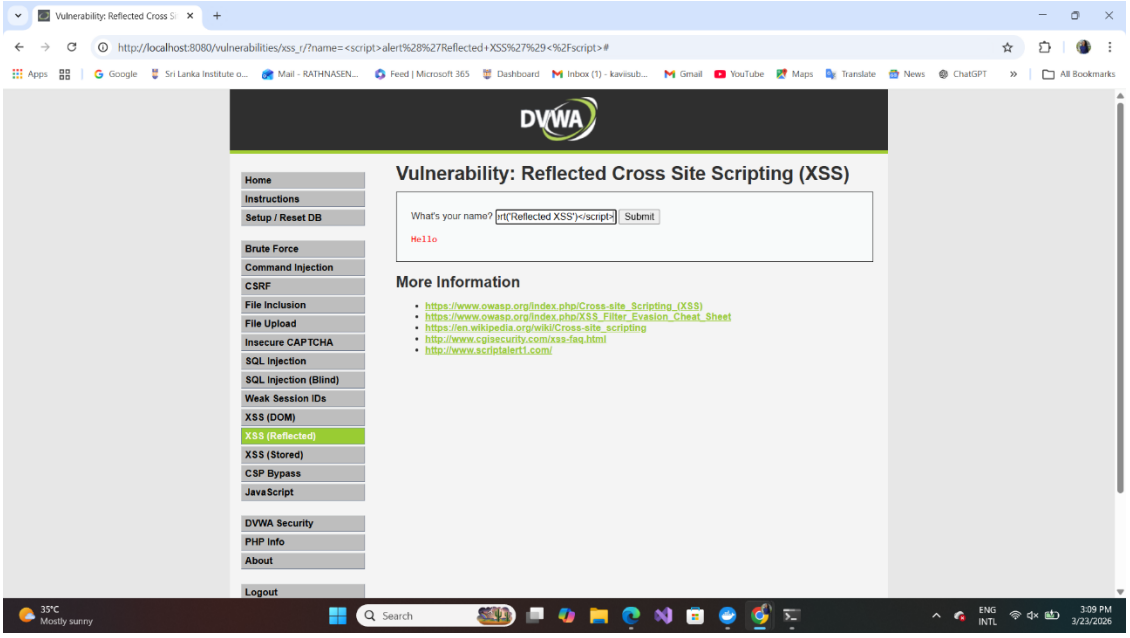
Severity

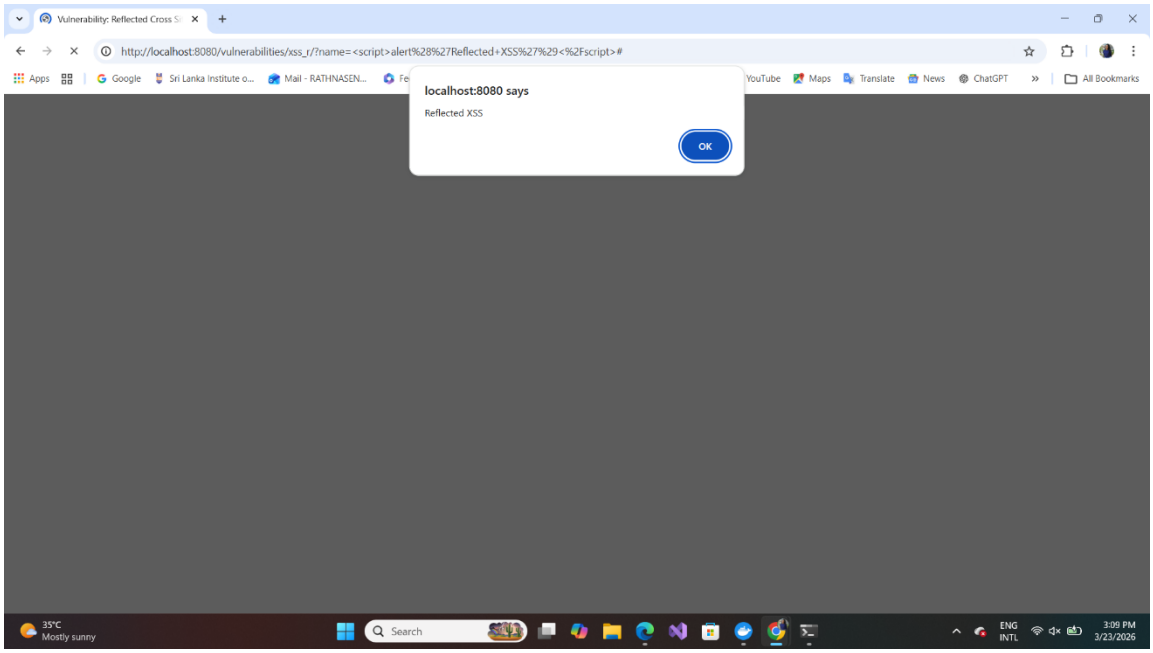
Critical

Mitigations

- **Avoid System Calls:** The best defense is to avoid using shell execution functions (like exec(), passthru(), or system()) entirely. Use built-in programming language APIs for tasks like pinging or file management.
- **Input Validation (Allowlisting):** If you must take an IP address, ensure the input matches a strict pattern (e.g., only numbers and dots). Reject any input containing special characters like ;, &, |, or $.
- **Escaping Shell Arguments:** Use specific functions (like escapeshellarg() in PHP) to wrap user input in quotes, ensuring the shell treats the entire input as a single string rather than a command.
- **Principle of Least Privilege:** Run the web application with a low-privileged user account that doesn't have permission to access sensitive system files or execute dangerous binaries.
-

4.5 Reflected Cross Site Scripting





In the "What's your name?" field, a JavaScript payload has been entered: <script>alert('Reflected XSS')</script>.

- **The Process:** When you click "Submit," the application takes whatever is in the input box and immediately "reflects" it back onto the page (e.g., "Hello [input]").

- **The Exploit:** Because the application doesn't sanitize the input, the browser thinks the script is part of the website's own code and executes the alert box.
- **The URL:** You can see the payload right in the browser's address bar. This is the hallmark of Reflected XSS—an attacker would send this exact link to a victim, and the script would run the moment the victim clicks it.

Impact

An attacker can steal a user's session cookie, redirect them to a phishing site, or perform actions on the site as if they were that user.

Severity

Medium to High

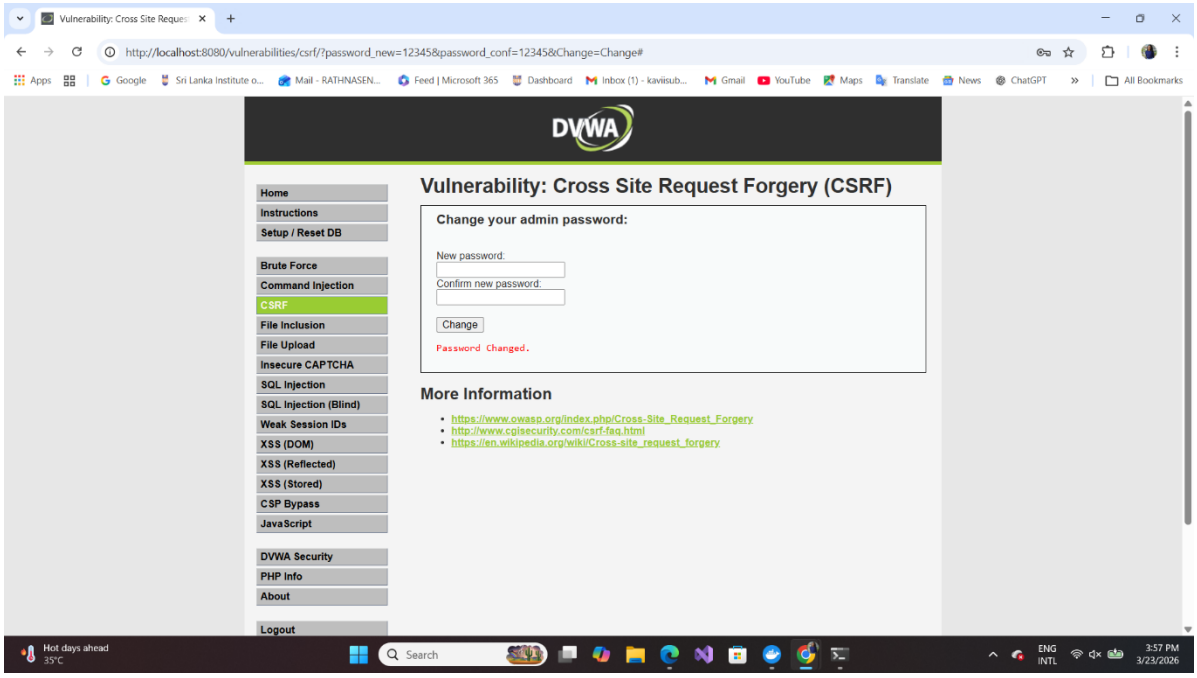
Mitigations

- **Output Encoding:** Before printing the user's name back to the screen, the server should convert characters like < and > into HTML entities *(< and >).* This prevents the browser from interpreting them as code.
- **Input Validation**: Ensure the input only contains expected characters (e.g., only letters for a name).
- **Use modern frameworks:** Many modern web frameworks (like React or Angular) automatically encode data by default, making this much harder to accidentally implement.

4.6 Cross Site Request Forgery (CSRF)

In a CSRF attack, an attacker tricks a victim's browser into performing an unwanted action on a different website where the victim is currently authenticated.

- **The Vulnerability:** The "Change your admin password" form does not use any unique tokens to verify that the request was intentionally made by the user.
- **The Exploit:** As seen in the URL bar, the password change is being handled via a **GET request** (?password_new=12345...). Because the browser automatically includes the user's session cookies with the request, an attacker can simply trick a logged-in admin into clicking a malicious link or visiting a site with a hidden "image" tag that points to this URL.
- **The Result:** The message **"Password Changed"** appears, confirming the action was successful without the user ever manually typing into the fields.



Impact

Attackers can perform state-changing actions on behalf of the user, such as changing passwords, transferring funds, or updating email addresses, leading to full account takeover.

Severity

High

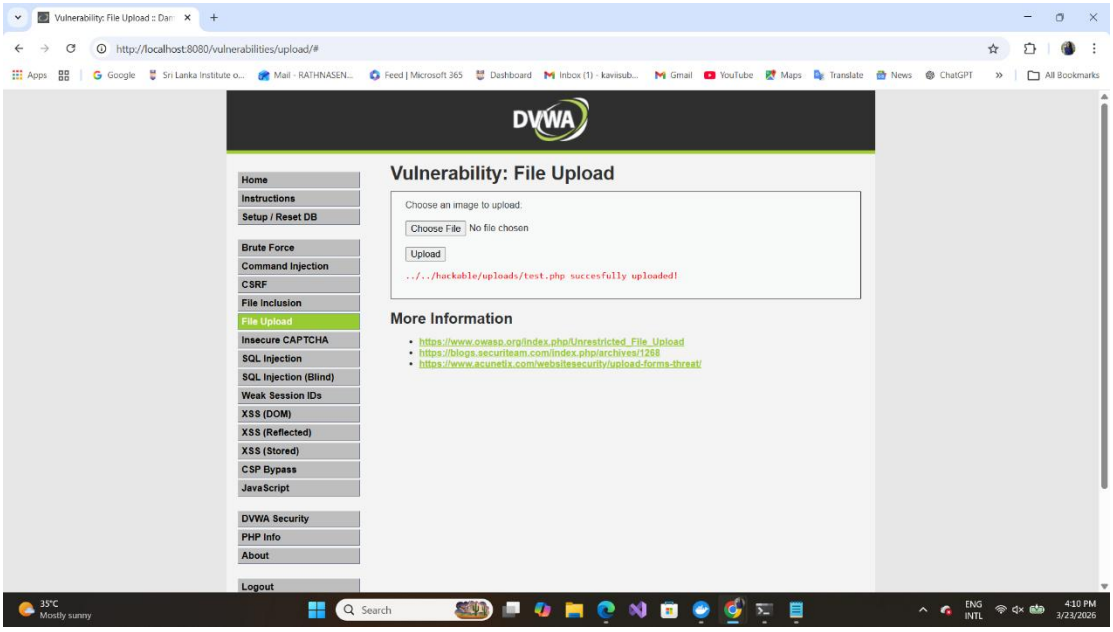
Mitigations

- **Anti-CSRF Tokens:** The most effective defense. The server should generate a unique, cryptographically strong, and random token for every session or request. The server then checks for this token before processing any sensitive action.
- **Use POST instead of GET:** Sensitive actions should never be handled via GET requests, as parameters are exposed in the URL and easily manipulated.
- **Re-authentication:** For critical actions (like changing a password), require the user to provide their *current* password or complete a CAPTCHA to prove intent.

4.7 Unrestricted File Upload

In this exercise, the "Choose an image to upload" feature is intended for image files (like .jpg or .png). However, as shown in the red success message: ../../hackable/uploads/test.php successfully uploaded!

- **The Exploit:** The user successfully uploaded a **PHP file** (test.php) instead of an image.
- **The Danger:** Because the server accepts .php files and stores them in a web-accessible directory, an attacker can now navigate to that file's URL. Once accessed, the server will execute the PHP code inside it. This is the first step in deploying a **Web Shell**, which allows an attacker to run remote commands on the server.



Impact

This often leads to **Remote Code Execution (RCE)**. An attacker can gain full control of the web server, steal the database, modify website content, or use the server as a jumping-off point to attack the internal network.

Severity

Critical

Mitigations

- **File Extension Filtering (Allowlisting):** Only allow specific, safe extensions (e.g., .jpg, .jpeg, .png). Never use a "blocklist" approach, as attackers can bypass it with extensions like .php5, .phtml, or .phar.
- **MIME Type Validation:** Check the file header (Magic Bytes) to ensure the file is actually what it claims to be (e.g., verifying that a .jpg file truly has an image structure).
- **Filename Randomization:** Rename uploaded files to a random string (e.g., a7f3k9.png) so attackers cannot easily guess the path to their uploaded script.
- **Store Files Outside Web Root**: Save uploaded files in a directory that is not directly accessible via a URL. Use a script to serve the files to users safely.
- **Disable Execution Permissions:** Configure the upload directory to prevent the web server from executing any scripts within it.

Conclusion

The security assessment conducted on the **Damn Vulnerable Web Application (DVWA)** successfully identified multiple critical and high-risk vulnerabilities within the application. These include SQL Injection, Command Injection, Stored and Reflected Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), authentication weaknesses, and unrestricted file upload.

The presence of these vulnerabilities demonstrates how improper input validation, insecure coding practices, and lack of security controls can expose a web application to serious threats. Several of the identified issues, such as SQL Injection, Command Injection, and File Upload vulnerabilities, could allow attackers to gain full control over the system, access sensitive data, or compromise user accounts.

This assessment highlights the importance of integrating security into every phase of the software development lifecycle. Even though DVWA is intentionally vulnerable for learning purposes, the same types of issues are commonly found in real-world applications, making this assessment highly relevant for practical cybersecurity understanding.

Recommendations

To improve the overall security posture of the application, the following recommendations are strongly advised:

1. Input Validation and Output Encoding

- Validate all user inputs using strict allowlisting techniques.
- Encode output properly to prevent XSS attacks.
- Reject any unexpected or malicious input.

2. Secure Coding Practices

- Use parameterized queries or prepared statements to prevent SQL Injection.
- Avoid executing system-level commands directly from user input.
- Follow secure development frameworks and standards.

3. Strong Authentication Mechanisms

- Implement account lockout policies after multiple failed login attempts.
- Introduce Multi-Factor Authentication (MFA).
- Use strong password policies and enforce complex requirements.

4. Protection Against CSRF

- Implement Anti-CSRF tokens for all sensitive operations.
- Use POST requests instead of GET for state-changing actions.
- Require re-authentication for critical actions.

5. Access Control and Authorization

- Enforce proper access controls to prevent unauthorized data access (e.g., IDOR).
- Apply the principle of least privilege across the system.

6. Security Configuration

- Regularly update and patch systems, frameworks, and libraries.
- Disable unnecessary services and features.
- Use secure headers and configurations.

7. Logging and Monitoring

- Implement logging mechanisms to track suspicious activities.
- Monitor login attempts and unusual behaviors.
- Use intrusion detection/prevention systems where applicable.

8. Regular Security Testing

- Conduct periodic penetration testing and vulnerability assessments.
- Use automated tools alongside manual testing for better coverage.
- Integrate security testing into CI/CD pipelines.